



FIREBRAND

LYCEUM EDUCATION GROUP

Events and Interaction

The day your code starts responding to a real person clicking a real button.

What today covers

By the end of today you will wire up buttons, inputs and page logic together.

1

Listening for clicks: using `addEventListener` so code runs only when a user acts

2

Reading input values: pulling text typed by a user out of an input field

3

Responding on the page: updating the DOM based on what a user entered

4

Reconnecting old logic: wiring earlier checkers up to a real form and button

5

Debugging event code: spotting and fixing common event-handling mistakes

Every button you have ever clicked

“

A button on a screen does nothing until code tells it what to do.

The idea behind today

From reading the page to reacting to the page

DAY 10: READING AND WRITING

Selected an element with `querySelector`

Read or changed `.textContent` and `.innerHTML`

Code ran once, top to bottom, when the page loaded

VS

TODAY: REACTING TO THE USER

Still selecting elements the same way

Still changing `.textContent` and `.innerHTML` the same way

Code now waits, and only runs when a user clicks

Meet addEventListener

script.js

```
1 const button = document.querySelector("#sayHello");  
2 button.addEventListener("click", function () {  
3   console.log("Button was clicked");  
4 });
```

1

The button

the element we want to watch for a click on

2

The event name

the word "click", written as a string

3

The function

the code that runs only when that click happens

Predict before we run it



predict-click.js

```
1 const button = document.querySelector("#checkButton");
2 button.addEventListener("click", function () {
3   console.log("Checking now");
4 });
5 console.log("Script finished loading");
```

VARIABLE	VALUE
First thing printed to the console	"Script finished loading"
What happens when the page first loads	nothing else, no click has happened yet
What happens after a user clicks the button	"Checking now" is printed

Trace it yourself

Predict what prints and in what order, before touching a keyboard.

- 1 Open predict-click-starter.js in your editor
- 2 Read the code without running it
- 3 Write down what prints, and in what order
- 4 Run it and click the button to check your prediction



TIMEBOX

8 minutes

Individual, compare with a neighbour for the last 2 minutes



OUTPUT

A short written prediction, checked against the real result

Where does the function actually live

The function inside `addEventListener` is written where it is used, not named and called separately.

Anonymous function

a function with no name, written directly inside the brackets

Still has parameters if needed

we will see this shortly with input values

Still has a body

the code between the curly braces runs when triggered

Still can use everything you know

if/else, comparisons, variables, all work inside it

A button on its own is not enough

Most real interactions need to know what the user typed, not just that they clicked.

01

An input field: a box on the page where a user types text

02

The `.value` property: how JavaScript reads what is currently typed in that box

03

Read only when needed: the value is fetched at the moment the button is clicked

04

Combines with everything you know: turn that text into a variable, then use it as normal

Reading a value from an input field

read-input.js

```
1 const nameInput = document.querySelector("#learnerName");
2 const button = document.querySelector("#greetButton");
3 button.addEventListener("click", function () {
4   const typedName = nameInput.value;
5   console.log("Hello, " + typedName);
6 });
```

1

Select the input

just like selecting any other element

2

Read .value

gives back whatever text is currently typed

3

Store it in a variable

now it behaves exactly like any other string

Reading input in action



read-input.js

```
1 const nameInput = document.querySelector("#learnerName");
2 const button = document.querySelector("#greetButton");
3 button.addEventListener("click", function () {
4     const typedName = nameInput.value;
5     console.log("Hello, " + typedName);
6 });
```

CONSOLE OUTPUT

Hello, Amara

Try reading a value yourself

Wire up a button that reads a typed name and prints a greeting.

- 1 Open greet-input-starter.js and its matching HTML file
- 2 Find the input id and button id already in the HTML
- 3 Complete the addEventListener so it reads .value and logs it
- 4 Test by typing a name and clicking the button

TIMEBOX

12 minutes
Pairs recommended

OUTPUT

A working script that greets whatever name is typed in

Two skills, one moment

Every interactive feature today follows the same three-part shape.

Listen: `addEventListener` waits for a click on a specific button

Read: `.value` pulls out whatever the user typed at that moment

Respond: existing logic, familiar from earlier days, decides what happens next

From console to page

Nobody using your website will ever open the console.

A reminder about who the output is really for

Updating the page instead of the console

greet-page.js

```
1 const nameInput = document.querySelector("#learnerName");
2 const button = document.querySelector("#greetButton");
3 const result = document.querySelector("#result");
4 button.addEventListener("click", function () {
5     const typedName = nameInput.value;
6     result.textContent = "Hello, " + typedName;
7 });
```

1

Select a result area

an element that already exists in the HTML, ready to hold a message

2

Build the message

exactly as before, using variables and concatenation

3

Write it to the page

.textContent replaces console.log

Move it from console to page

Change your greeting script so the message appears on the page.

CASE FILE — ACTIVITY BRIEFING

OBJECTIVE

A greeting that appears visibly on the page, not just the console

BRIEFING STEPS

- 01** Open your completed greet-input-starter.js from before
- 02** Add a line selecting the #result element
- 03** Replace console.log with result.textContent
- 04** Test by typing a name and clicking the button

 10 minutes

Two ways to see if a click really happened

console.log approach

- Message only visible to the developer
- Useful while building and debugging
- Disappears the moment the console closes

.textContent approach

- Message visible to anyone using the page
- What a real user actually experiences
- Persists on the page until changed again

Bringing back familiar logic

The if/else checkers from earlier weeks slot straight into a click handler.

0

The condition itself

unchanged, the same comparisons learners have always used

1

The values being compared

now come from `.value` instead of a fixed variable

2

The output

written to the page with `.textContent` instead of `console.log`

3

The trigger

a click, instead of the script simply running top to bottom

4

The pass checker, revisited

pass-checker-event.js

```
1 const scoreInput = document.querySelector("#scoreInput");
2 const button = document.querySelector("#checkButton");
3 const result = document.querySelector("#result");
4 button.addEventListener("click", function () {
5   const score = scoreInput.value;
6   if (score >= 50) {
7     result.textContent = "Pass";
8   } else {
9     result.textContent = "Fail";
10  }
11 });
```

1

Same comparison as Day 2

score against a pass mark

2

New source

the score now comes from a typed value

3

New destination

the result is written to the page

Spot what changed and what stayed the same

Compare today's pass checker with the Day 2 version side by side.

- Instructions Open pass-checker-event.js and your old pass-checker.js from Day 2 List which lines are genuinely new List which lines are unchanged in logic Discuss with a partner why each new line was needed
- Timebox 10 minutes
- Output A short written list of new versus unchanged lines

A quiet truth about today



Nearly everything today is old logic wearing a new trigger.

The morning build, a real eligibility form

A savings account provider wants their eligibility checker on a real web page.

1

CONTEXT

The eligibility checker from Day 4 currently only runs with a fixed score written into the script.

2

CHALLENGE

A real applicant needs to type their own details and click a button to see the result.

3

TASK

Wire the existing eligibility logic up to a real input field and button.

The starter file for the morning build

This is what you will find open in morning-build-starter.js.

morning-build-starter.js (HTML, provided)

```
1 <input id="ageInput" type="text" />
2 <button id="checkButton">Check eligibility</button>
3 <p id="result"></p>
```

morning-build-starter.js (JavaScript, provided)

```
1 const ageInput = document.querySelector("#ageInput");
2 const button = document.querySelector("#checkButton");
3 const result = document.querySelector("#result");
```

Build the eligibility form

Wire the Day 4 eligibility logic up to a real button and input.

- 1 Open morning-build-starter.js, HTML and JavaScript files
- 2 Add addEventListener for a click on the button
- 3 Read ageInput.value inside the click handler
- 4 Reuse the Day 4 if/else logic to decide the result
- 5 Write the result to the page using result.textContent



TIMEBOX

18 minutes

Test with at least two different ages before finishing



OUTPUT

A working eligibility form showing Eligible or Not eligible on the page

What did you notice building that

Morning recap

Three new pieces of syntax, endless reuse of everything you already know.

1

addEventListener: waits for a click, then runs a function you provide

2

.value: reads whatever text a user has currently typed into an input

3

Old logic, new home: if/else, comparisons and messages transfer straight across



Lunch

Back for the afternoon's main project after a well-earned break.



FIREBRAND

LYCEUM EDUCATION GROUP

Afternoon: validating what a user actually types

This afternoon, we make sure the page copes with messy, real-world input.

What a user actually types

“

Users will type anything, including nothing at all.

A lesson every developer learns early

What happens with an empty input



empty-input-check.js

```
1 const nameInput = document.querySelector("#learnerName");
2 const button = document.querySelector("#greetButton");
3 const result = document.querySelector("#result");
4 button.addEventListener("click", function () {
5     const typedName = nameInput.value;
6     result.textContent = "Hello, " + typedName;
7 });
```

CONSOLE OUTPUT

Hello,

Guarding against an empty input

guarded-greet.js

```
1 button.addEventListener("click", function () {  
2   const typedName = nameInput.value;  
3   if (typedName === "") {  
4     result.textContent = "Please type your name first";  
5   } else {  
6     result.textContent = "Hello, " + typedName;  
7   }  
8 });
```

1

Check first

is the typed value actually empty

2

Branch the message

give a helpful prompt instead of a broken greeting

3

Only greet properly

when there is genuinely something to greet

Add the empty-input guard yourself

Add a guard so an empty stock code shows a helpful message.

CASE FILE — ACTIVITY BRIEFING

OBJECTIVE

A stock lookup that prompts for input rather than showing a blank result

BRIEFING STEPS

- 01** Open stock-lookup-starter.js in your editor
- 02** Find the existing addEventListener block
- 03** Add an if check for an empty string before the lookup message
- 04** Test with an empty box and then with real text typed in

 10 minutes

Beyond empty, checking a sensible range

Just like Day 2's pass checker, some values need to sit within a valid range.

Empty check first: catch nothing-typed-at-all before anything else

Range check second: catch values that are technically present but invalid

Clear feedback always: tell the user exactly what went wrong, not just that it did

A validated score checker, wired to a form

validated-score-check.js

```
1 button.addEventListener("click", function () {
2   const score = scoreInput.value;
3   if (score === "") {
4     result.textContent = "Please enter a score";
5   } else if (score < 0 || score > 100) {
6     result.textContent = "Score must be between 0 and 100";
7   } else if (score >= 50) {
8     result.textContent = "Pass";
9   } else {
10    result.textContent = "Fail";
11  }
12 });
```

1

First guard

catches a blank submission

2

Second guard

catches an out-of-range number typed as text

3

Final branch

only reached once both guards are satisfied

Trace the validated checker

validated-score-check.js

```
1 if (score === "") {  
2   result.textContent = "Please enter a score";  
3 } else if (score < 0 || score > 100) {  
4   result.textContent = "Score must be between 0 and 100";  
5 } else if (score >= 50) {  
6   result.textContent = "Pass";  
7 } else {  
8   result.textContent = "Fail";  
9 }
```

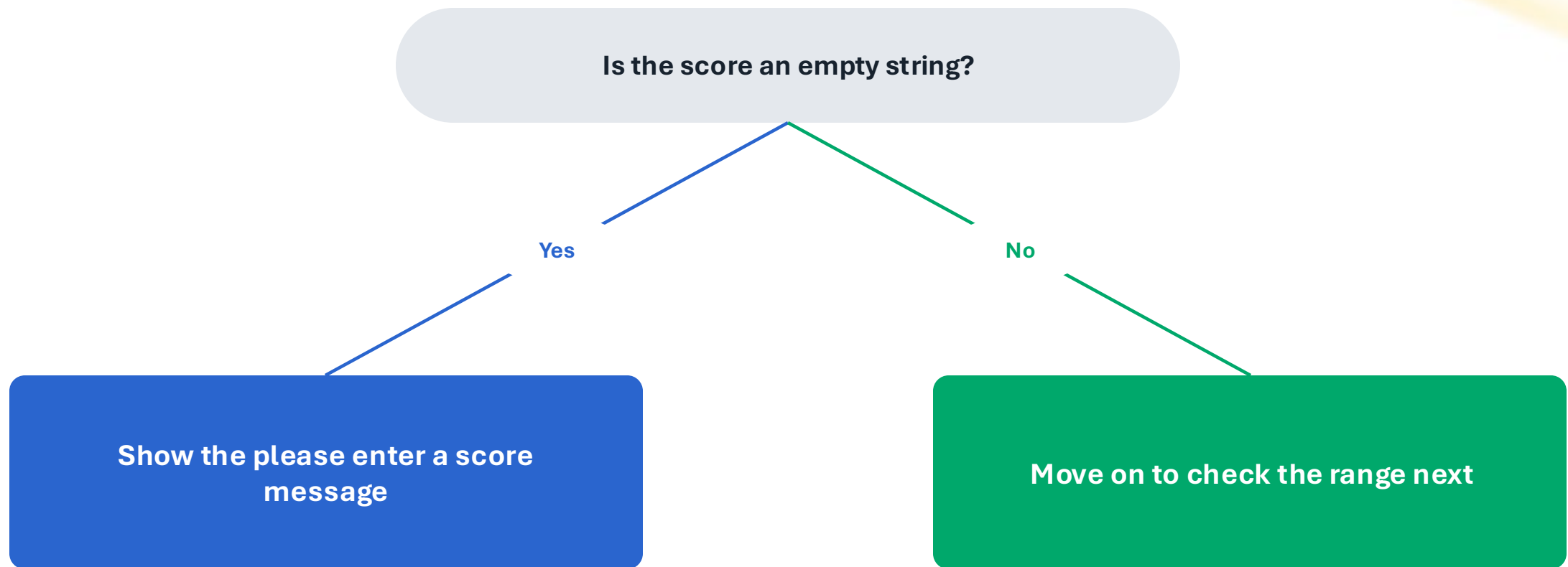
VARIABLE	VALUE
score is ""	result becomes "Please enter a score"
score is "150"	result becomes "Score must be between 0 and 100"
score is "42"	result becomes "Fail"
score is "75"	result becomes "Pass"

Build the validated checker yourself

Extend a single-guard checker into a full four-branch validated version.

- Instructions Open `extend-validation-starter.js` in your editor It currently only has the empty-input guard
- Add the range check using `else if` and logical `OR` Add the final pass or fail branches Test with an empty box, an out-of-range number, and a valid number
- Timebox 15 minutes Pairs recommended
- Output A fully validated checker matching the pattern just demonstrated

Why order matters in a branch chain



Something is broken here

This script looks right at a glance, but clicking the button does nothing.



broken-click.js

```
1 const input = document.querySelector("#scoreInput")
2 const button = document.querySelector("#checkButton");
3 const result = document.querySelector("#result");
4 button.addEventListener("click", function () {
5     result.textContent = "Score received: " + input.value;
6 });
```

Find and fix the bug

Debug a script where clicking the button appears to do nothing.

- 1 Open broken-click.js from the previous slide in your editor
- 2 Run it and click the button, notice nothing happens
- 3 Check the browser console for an error message
- 4 Read the error carefully and compare it against a working example
- 5 Fix the bug and confirm the button now works



TIMEBOX

10 minutes



OUTPUT

**A corrected script where
clicking the button updates the
result text**

The habits that catch bugs early

Lead argument

Clear

Read the console error message before changing anything It usually names the exact problem and where it is

Point 1

Clear

Compare against a known working example Spotting a difference is often faster than re-reading from scratch

Point 2

Flag

Assuming the newest line you wrote is always the bug Sometimes an earlier, unrelated line is the real cause

Point 3

Flag

Changing several things at once while debugging Makes it hard to tell which change actually fixed it

The afternoon build, an access request form

A critical infrastructure provider wants a simple access-level checker on their internal portal.

1

CONTEXT

Staff request access to a system by typing a clearance level and clicking a button.

2

CHALLENGE

The form must handle a blank submission, an invalid clearance level, and a valid one, each with a clear message.

3

TASK

Build a validated, event-driven checker following the listen, read, respond pattern.

The starter file for the afternoon build

This is what you will find open in afternoon-build-starter.js.

afternoon-build-starter.js (HTML, provided)

```
1 <input id="clearanceInput" type="text" />
2 <button id="checkAccessButton">Check access</button>
3 <p id="result"></p>
```

afternoon-build-starter.js (JavaScript, provided)

```
1 const clearanceInput = document.querySelector("#clearanceInput");
2 const button = document.querySelector("#checkAccessButton");
3 const result = document.querySelector("#result");
```

Build the access request checker

Combine everything today into one validated, event-driven checker.

- 1 Open afternoon-build-starter.js, HTML and JavaScript files
- 2 Add addEventListener for a click on the button
- 3 Read clearanceInput.value inside the click handler
- 4 Add a guard for an empty submission
- 5 Add branches for standard, elevated, restricted and unrecognised values

TIMEBOX

20 minutes

Test all five cases before finishing

OUTPUT

A working access checker handling blank, invalid and three valid clearance levels

Checking your build against the brief

Clear

Blank input shows a please enter a value message Confirm this by clicking with nothing typed in the box

Clear

An unrecognised value shows a clear rejection message Try typing something like guest or admin to confirm

Clear

Each of the three valid levels shows its own distinct message Test standard, elevated and restricted individually

Flag

Forgetting to test after every new branch added Small typos are far easier to catch one branch at a time

Two valid ways to write the same check

Both versions correctly identify the same three clearance levels.

version-a.js

```
1  if (clearance === "standard") {
2    result.textContent = "Standard access granted";
3  } else if (clearance === "elevated") {
4    result.textContent = "Elevated access granted";
5  } else if (clearance === "restricted") {
6    result.textContent = "Restricted access granted";
7  } else {
8    result.textContent = "Unrecognised clearance level";
9  }
```

version-b.js

```
1  if (clearance === "standard" || clearance === "elevated" || clearance === "restricted") {
2    result.textContent = clearance + " access granted";
3  } else {
4    result.textContent = "Unrecognised clearance level";
5  }
```

Compare your build to these two versions

Decide which of the two versions your own build most closely resembles.

CASE FILE — ACTIVITY BRIEFING

OBJECTIVE

A short spoken or written comparison of your approach against both versions

BRIEFING STEPS

- 01** Reread your completed afternoon-build-starter.js
- 02** Compare it against version-a.js and version-b.js
- 03** Note which one it is closer to, and one difference
- 04** Discuss with a neighbour which style you each prefer

 8 minutes

Explain your checker to two audiences

Practise the two-audience explanation skill from Day 3 on today's build.

- Instructions Write two sentences explaining your checker to a fellow developer Write two sentences explaining it to a non-technical colleague Compare the language choices between the two versions Share one version with a partner
- Timebox 10 minutes
- Output Two short written explanations of the same checker for different audiences

Where this pattern goes from here

Today's listen, read, respond pattern is the foundation for almost every interactive feature.

Forms with several fields

reading multiple `.value` properties in one click handler

Buttons that trigger different actions

multiple `addEventListener` calls on one page

Live feedback as a user types

listening for other events beyond click

Validated multi-step processes

chaining several checks before a final action

One more context, a delivery status flag

The same listen, read, respond shape, wrapped around a logistics scenario.



delivery-flag-starter.js

```
1 const trackingInput = document.querySelector("#trackingInput");  
2 const button = document.querySelector("#checkDeliveryButton");  
3 const result = document.querySelector("#result");
```

Build the delivery tracking check

Apply today's full pattern once more in a logistics context.

- 1 Open delivery-flag-starter.js, HTML and JavaScript files
- 2 Add addEventListener for a click on the button
- 3 Add a guard for an empty tracking number
- 4 Add a check using trackingInput.value.length for length exactly 6
- 5 Write each result to the page using result.textContent



TIMEBOX

12 minutes



OUTPUT

**A tracking checker handling
blank, invalid-length and valid
submissions**

What made today click

The first time your own code responds to your own click is a genuinely good feeling.

Worth naming, not just assuming

Today in your own words

Exit ticket

A short written wrap-up capturing today's key ideas in your own words.

CASE FILE — ACTIVITY BRIEFING

OBJECTIVE

A four-part written exit ticket

BRIEFING STEPS

- 01** Write one sentence explaining what `addEventListener` does
- 02** Write one sentence explaining what `.value` reads
- 03** Name one bug you fixed or guard you added today
- 04** Name one thing you would like more practice with

 **10 minutes**
Individual,
written

Session close

Today, your code finally started listening.

- ✓ addEventListener waits for a real user action before running your code
- ✓ .value reads exactly what a user has typed, at the moment they click
- ✓ Old logic, if/else, comparisons, validation, transfers straight into a click handler
- ✓ Every build today followed one repeatable shape: listen, read, respond